EECE 568 - Assignment 3

Matthew Yedlin, Bhushan Gopaluni

Due Date: November 19, 2026 - 11:59 pm

Late submissions will not be accepted. In case of an emergency, please inform the instructors.

Deliverables: Please upload the completed version of the jupyter notebook file (.ipynb) on Canvas. You need to complete the sections marked with a *ToDo* tag in the notebook. The jupyter notebook that you submit must contain the cell output from a clean execution (restart kernel and run all cells sequentially).

Runtime: This assignment includes only PyTorch coding questions. You will need a GPU, for which we recommend using Google Colaboratory. You can easily unzip the assignment3 folder containing a jupyter notebook and a CSV file. Upload the folder on your google drive and run the notebook.

1 Text Classification using Transformers - 10 points

The IMDB dataset contains 50000 movie reviews. Reviews are labeled as either positive (1) or negative (0). We want to implement a Transformer encoder that takes each review (series of words) as input and predicts whether it is a positive or a negative review.

1.1 Load and Tokenize Data

We use a preprocessed version of the IMDB dataset done by Affandy Fahrizain, which is attached to your assignment folder. We then assign a unique token (number) to each word, so we can apply mathematical operations to them.

1.2 Dataloader

The dataloader returns a Python dictionary containing reviews and their labels. Also, the data is split into train, validation, and test sets.

1.3 Network (5 pts)

Implement your network based on the encoder block of Transformers [link]. You can write your network from scratch or use the implemented versions. Either way, you need to understand the implementation and have access to the attention weights and the word embeddings. Also, your network should take a python dictionary as input and return a dictionary as output. More details in the jupyter notebook.

1.4 Training

The training script is provided to you similar to assignment 2. Feel free to play around with hyperparameters, but **train the model for exactly 10 epochs for a fair comparison**.

1.5 Evaluation (5 pts)

Report the model's accuracy on the train, validation, and test sets. Also, plot the attention weights as a heatmap between the cls token and the tokens of a random movie review from the test set. Moreover, plot the word embedding vectors in 2D. To do that, we use UMAP, which is a machine-learning technique for visualizing high-dimensional data by mapping it to a low-dimensional space.

2 Image Classification using Vision Transformers - 6 points

In this question, we want to use Vision Transformer (ViT) to do image classification on the MNIST dataset, a collection of 28×28 images of handwritten digits from 0 to 9. The network takes one image as input and predicts the digit it represents.

2.1 Dataset and Dataloader

The dataloader returns a Python dictionary containing images and their labels. Also, the data is split into train, validation, and test sets.

2.2 Network (2 pts)

Implement a simple Vision Transformer [link1] [link2]. You can write your network from scratch or use the implemented versions. Preferably, you can start from your model in Q1, add patch-embedding layers, add a class token, and build up toward a vision transformer. Ultimately, your network should take a Python dictionary as input and return a dictionary as output. More details in the jupyter notebook.

2.3 Training

The training script is provided to you. Feel free to play around with hyperparameters, but **train the model for exactly 10 epochs for a fair comparison**.

2.4 Evaluation (4 pts)

Report the model's accuracy on the train, validation, and test sets. Also, plot your network's input images and output for a few samples in the test set. Also, overlay the attention weights between the cls token and image tokens, and plot the overlayed images.

3 Multi-Task Vision Transformer - 4 points

Each image from the MNIST dataset is vertically flipped in this question with a probability of 0.5. Therefore, in addition to the "digit" label, each image has an "is_flipped" label, which is 1 when the image is inverted and 0 otherwise. We want to predict both of these labels simultaneously.

3.1 Dataset and Dataloader

The dataloader returns a Python dictionary containing images and their labels. Also, the data is split into train, validation, and test sets.

3.2 Network (2 pts)

Start from the model in Q2, add another class token for predicting the "is_flipped" label, and apply a new MLP head on this class token. Your network would be very similar to Q2, but instead, it should have two branches, each with its loss function. **Your network should consist of only one encoder block.**

Ultimately, your network should take a Python dictionary as input and return a dictionary as output. More details in the jupyter notebook.

3.3 Training

The training script is provided to you. Feel free to play around with hyperparameters, but **train the model for exactly 10 epochs for a fair comparison**. You can also use a weighted sum of each branch's loss to calculate the total loss function.

3.4 Evaluation (2 pts)

Report the model's accuracy on the train, validation, and test sets. Also, plot your network's input images and output for a few samples in the test set.